Unidad 1. Introducción a la programación con frameworks

Programación de aplicaciones utilizando frameworks · 2.º DAW

IES Hontanilla · Tarancón

Curso 2026-2027

Índice general

Presentación de la unidad **3**

Introducción . . . 3

Resultados de aprendizaje y criterios de evaluación . . . 3

Temporalización . . . 4

Entorno de trabajo . . . 4

Convenciones de este documento . . . 4

1 Los frameworks **5**

Para empezar . . . 5

1.1 El punto de partida: tareas que se repiten . . . 5

1.2 Qué es un framework . . . 6

1.2.1 La inversión de control . . . 6

1.3 Librería y framework . . . 7

1.4 Ventajas e inconvenientes . . . 7

1.5 Arquitectura: el patrón Modelo-Vista-Controlador . . . 7

1.5.1 MTV: la versión de Django . . . 8

1.6 Panorama actual . . . 8

1.6.1 Criterios para elegir un framework . . . 9

Ejercicio 1.1. Investigación sobre un framework . . . 9

2 Instalación del entorno de trabajo **11**

Para empezar . . . 11

2.1 Qué se va a instalar . . . 11

2.2 Instalación de Python . . . 12

2.2.1 La terminal PowerShell . . . 12

2.2.2 Comprobar si ya hay Python . . . 12

2.2.3 Descarga e instalación . . . 12

Ejercicio 2.1. Comprobación de la instalación de Python 13
2.3 Visual Studio Code 13
2.4 La carpeta de trabajo y el entorno virtual 13
2.4.1 Por qué un entorno virtual 13
2.4.2 Carpeta de trabajo 13
2.4.3 Permitir la activación del entorno en PowerShell 14
Ejercicio 2.2. Creación del entorno virtual e instalación de Django 14
2.5 Creación del proyecto Django 15
Ejercicio 2.3. Arranque del servidor de desarrollo 16
2.6 Problemas frecuentes 16

3 Primer proyecto: sin framework y con framework 18

Para empezar 18
3.1 La misma página, dos veces 18
3.2 Sin framework: el servidor de la biblioteca estándar 18
Ejercicio 3.1. Servidor HTTP sin framework 19
3.3 Con framework: proyecto y aplicación Django 20
3.3.1 Proyecto y aplicaciones 20
3.3.2 Rutas y vistas 20
Ejercicio 3.2. Primera aplicación Django 21
3.4 Comparación 21

4 Cierre de la unidad 23

4.1 Resumen 23
4.2 Glosario 23
4.3 Preguntas de autoevaluación 24
4.4 Referencias 24

Presentación de la unidad

Introducción

Casi todas las aplicaciones web que se utilizan a diario resuelven los mismos problemas internos: reciben peticiones HTTP, deciden qué código debe atenderlas, comprueban quién es el usuario, validan los datos de los formularios, consultan una base de datos y devuelven una respuesta. Programar todo eso desde cero en cada proyecto es lento y propenso a errores. Los **frameworks** existen para no tener que hacerlo.

Esta unidad presenta el módulo y los conceptos que se utilizarán durante todo el curso. Tiene tres apartados:

1. **Los frameworks.** Qué es un framework, en qué se diferencia de una librería, qué ventajas e inconvenientes tiene, cómo organiza una aplicación (patrón Modelo-Vista-Controlador) y qué frameworks se usan hoy en cada capa de una aplicación web.
2. **Instalación del entorno de trabajo.** Instalación de Python y de Django en Windows, partiendo de un equipo sin nada instalado, con entornos virtuales.
3. **Primer proyecto: sin framework y con framework.** La misma página programada dos veces, primero a mano y después con Django, para comprobar en la práctica qué aporta un framework.

Resultados de aprendizaje y criterios de evaluación

La unidad inicia el resultado de aprendizaje 1, que se desarrolla por completo en la Unidad 2.

RA	Descripción	Peso en la nota final
RA1	Crea aplicaciones BackEnd utilizando framework de servidor	22 %

Criterio	Contenido	Apartado
RA1.a	Estructura base de un proyecto creado con un framework de servidor	2.5, 3.3, 3.4
RA1.b	Creación de rutas y controladores (primer contacto)	3.3

Temporalización

Sesiones	Contenido
S01–S02	Apartado 1. Los frameworks
S03–S04	Apartados 2 y 3. Instalación y primer proyecto

Entorno de trabajo

El módulo se trabaja directamente sobre **Windows 10 u 11**, sin máquinas virtuales, con estas herramientas:

Herramienta	Uso	Unidad
Visual Studio Code	Editor de código	Todas
Python 3.13 y Django 5.2 LTS	Framework de servidor	1 y 2
Node.js 24 LTS y Vue 3	Framework de cliente	3
Docker Desktop	Despliegue en contenedores	4
PrimeVue	Framework de diseño de interfaces	5
pytest, Vitest y Playwright	Frameworks de pruebas	6

Convenciones de este documento

- El texto en `fuente monoespaciada` es lo que se escribe tal cual: comandos, nombres de fichero, direcciones.
- Los comandos se escriben en la terminal **PowerShell** de Windows. Cuando un bloque muestra un comando precedido de `>`, el carácter `>` no se escribe: representa el indicador de la terminal.
- Cada ejercicio termina con un apartado **Verificación**, que indica el resultado exacto que se debe obtener. No se pasa al ejercicio siguiente hasta obtenerlo.
- Los bloques **Para reflexionar** plantean una pregunta para pensar o comentar en clase; no tienen una única respuesta correcta.

Capítulo 1

Los frameworks

Para empezar

Antes de cualquier definición, conviene pensar en estas tres preguntas:

1. ¿Qué tienen en común, por dentro, las aplicaciones web que utilizáis a diario?
2. ¿Qué partes de una aplicación web aparecen en casi todos los proyectos, sea cual sea su temática?
3. Si habéis oído la palabra *framework*, ¿cómo la definiríais con vuestras palabras?

1.1 El punto de partida: tareas que se repiten

Una tienda en línea, una red social, la plataforma EducamosCLM o la aplicación de un banco son muy distintas para quien las usa, pero por dentro realizan casi las mismas tareas:

Tarea	En qué consiste
Enrutar peticiones HTTP	Decidir qué parte del código atiende cada dirección (`/productos`, `/login`…)
Gestionar sesiones de usuario	Recordar quién es el usuario entre una petición y la siguiente
Validar datos de formularios	Comprobar que lo que envía el usuario es correcto antes de guardarlo
Acceder a la base de datos	Guardar, consultar, modificar y borrar información
Autenticar y autorizar	Comprobar quién es el usuario (autenticación) y qué puede hacer (autorización)
Gestionar errores y registros	Responder de forma controlada cuando algo falla y dejar constancia en un registro (*log*)

Programar estas tareas de nuevo en cada proyecto tiene tres problemas: consume mucho tiempo, cada equipo las resuelve de una manera distinta y, sobre todo, es fácil cometer errores de seguridad. Un fallo al validar un formulario o al comprobar una contraseña puede comprometer toda la aplicación.

Para reflexionar. *¿Tiene sentido volver a programar estas tareas en cada proyecto? ¿Qué alternativas se os ocurren?*

1.2 Qué es un framework

Un **framework** (en español, *marco de trabajo*) es un conjunto de componentes, convenciones y herramientas que proporciona la **estructura base** de una aplicación. El desarrollador completa esa estructura con el código propio del problema que quiere resolver.

El framework aporta, ya programadas y probadas, las tareas comunes de la tabla anterior. El desarrollador escribe solo lo que es específico de su aplicación: qué datos maneja, qué reglas de negocio aplica y qué páginas muestra.

1.2.1 La inversión de control

El rasgo que mejor caracteriza a un framework es la **inversión de control**: es el framework quien gestiona el flujo de ejecución del programa y quien **invoca el código del desarrollador** cuando corresponde. El desarrollador no decide cuándo se ejecuta su código; decide qué hacer cuando el framework lo llame.

El siguiente ejemplo, escrito con el framework Django (Python), define una página que responde «Hola» en la dirección `/hola/`:

```python
# urls.py
from django.urls import path

from . import views

urlpatterns = [
    path("hola/", views.hola),
]
```

```python
# views.py
from django.http import HttpResponse


def hola(request):
    return HttpResponse("Hola")
```

En este código nadie llama a la función `hola()`. Cuando llega una petición a `/hola/`, Django recorre la lista `urlpatterns`, encuentra la ruta que coincide y llama a `hola()` pasándole un objeto con los datos de la petición (`request`). La función solo tiene que devolver la respuesta.

Esta idea se conoce también como **principio de Hollywood**: «no nos llames; ya te llamaremos nosotros».

Para reflexionar. *En el código anterior nadie llama a `hola()`. ¿Quién la ejecuta, y cuándo?*

1.3 Librería y framework

Una **librería** (o biblioteca) también es código reutilizable, pero la relación con el programa es la contraria: es el programa quien llama a la librería cuando la necesita, y el programa sigue teniendo el control.

	Librería	Framework
Control del flujo	El programa llama a la librería	El framework llama al programa
Estructura	La decide el desarrollador	La define el framework
Alcance	Una tarea concreta	La aplicación completa
Ejemplos	jQuery, Gson, Chart.js	Spring Boot, Angular, Django

La frontera no siempre es nítida. **React** se presenta a sí mismo como una librería para construir interfaces, aunque impone una forma de trabajar y suele usarse junto con otras piezas que forman un marco completo. **Bootstrap** se presenta como un framework de diseño, aunque no invierte el control del programa: lo que aporta es una estructura de clases y componentes visuales. En la práctica, la palabra *framework* se usa en sentido amplio para cualquier conjunto de herramientas que impone una estructura de trabajo.

> ***Para reflexionar.*** *Bootstrap se presenta como framework y React como librería. Con la tabla anterior, ¿estáis de acuerdo con esas etiquetas?*

1.4 Ventajas e inconvenientes

Ventajas	Inconvenientes
Menos código repetitivo: las tareas comunes ya están resueltas	Curva de aprendizaje inicial: hay que conocer sus convenciones
Estructura común y predecible: cualquier desarrollador que conozca el framework entiende el proyecto	Dependencia de sus decisiones y de sus versiones: una actualización puede obligar a cambiar código
Mecanismos de seguridad ya probados por miles de proyectos	Comportamiento oculto tras la abstracción: cuando algo falla, cuesta saber por qué
Documentación y comunidad amplias	Sobrecarga en proyectos muy pequeños

> ***Para reflexionar.*** *¿En qué tipo de proyecto no compensaría usar un framework?*

1.5 Arquitectura: el patrón Modelo-Vista-Controlador

La mayoría de los frameworks de servidor organizan la aplicación siguiendo el patrón **Modelo-Vista-Controlador** (MVC), que separa tres responsabilidades:

Componente	Responsabilidad	En Spring Boot	En Django
Controlador	Atender la petición y coordinar la respuesta	Clases `@Controller`	`urls.py` y `views.py`
Modelo	Datos, persistencia y reglas de negocio	Clases `@Entity` y JPA	`models.py` (ORM)
Vista	Representación de la respuesta	Plantillas Thymeleaf o JSON	Plantillas (*templates*)

El recorrido de una petición como `GET /productos` es el siguiente:

1. El **controlador** recibe la petición y decide qué hay que hacer.
2. El controlador pide al **modelo** los datos necesarios (por ejemplo, la lista de productos). El modelo los obtiene de la base de datos y aplica las reglas de negocio.
3. El controlador entrega esos datos a la **vista**, que construye la respuesta (una página HTML o un documento JSON).
4. El controlador devuelve la respuesta al cliente.

El modelo y la vista no se comunican entre sí: es el controlador quien los coordina. Esta separación permite cambiar la presentación sin tocar los datos, probar cada parte por separado y repartir el trabajo en un equipo.

1.5.1 MTV: la versión de Django

Django usa el mismo patrón con otros nombres: **MTV** (*Model-Template-View*).

- El **modelo** (*Model*) es el mismo concepto que en MVC.
- La **plantilla** (*Template*) es lo que MVC llama vista: la presentación.
- La **vista** (*View*) de Django es la función o clase que atiende la petición, es decir, lo que MVC llama controlador.

Además, el propio framework hace parte del trabajo del controlador: el sistema de rutas (`urls.py`) decide qué vista atiende cada petición.

> ***Para reflexionar.*** *¿Qué se gana al separar estas tres responsabilidades? Django lo llama MTV: ¿por qué creéis que eligió esos nombres?*

1.6 Panorama actual

Una aplicación web moderna tiene varias capas, y cada una tiene sus propios frameworks:

Capa	Ejemplos	Lenguaje
Servidor (*back-end*)	Spring Boot, Laravel, Django, NestJS	Java, PHP, Python, TypeScript
Cliente (*front-end*)	Angular, React, Vue	TypeScript, JavaScript
Diseño de interfaces	Bootstrap, Tailwind CSS, PrimeVue	CSS, JavaScript
Persistencia	Hibernate (JPA), Eloquent, ORM de Django	Java, PHP, Python
Pruebas	JUnit, Mockito, pytest, Jest, Vitest, Playwright	Java, Python, JavaScript

En este módulo se utilizará una pila completa y coherente:

- **Django** como framework de servidor (Unidad 2).
- **Vue** como framework de cliente (Unidad 3).
- **Docker** para el despliegue (Unidad 4).
- **PrimeVue** como framework de diseño (Unidad 5).
- **pytest, Vitest y Playwright** para las pruebas (Unidad 6).

1.6.1 Criterios para elegir un framework

No existe un framework mejor que los demás para todo. Al elegir uno para un proyecto real conviene tener en cuenta:

- **El lenguaje** que conoce el equipo.
- **La madurez y el soporte**: quién lo mantiene, cada cuánto publica versiones y si tiene versiones con soporte extendido (LTS, *Long Term Support*).
- **La licencia**: si permite el uso comercial.
- **La comunidad y la documentación**.
- **El tipo de proyecto**: una API, una web con muchas páginas, una aplicación en tiempo real…
- **El mercado laboral** de la zona.

> ***Para reflexionar.*** *¿Por qué existen tantos frameworks para resolver lo mismo? ¿Qué criterios usaríais vosotros para elegir uno?*

Ejercicio 1.1. Investigación sobre un framework

Objetivo: conocer un framework real a partir de su documentación oficial.

1. Elegid un framework de la tabla del panorama actual.
2. Responded a estas preguntas citando la documentación oficial (con enlace):
    1. ¿Qué problema principal resuelve y para qué capa está pensado?
    2. ¿Sigue el patrón MVC u otra arquitectura? ¿Cuál?

   3. ¿Quién lo mantiene, bajo qué licencia y cada cuánto publica versiones?
   4. ¿Qué hace falta instalar para crear un primer proyecto?

3. Preparad una exposición de 5 minutos para la clase.

Verificación. Cada respuesta incluye al menos un enlace a la documentación oficial del framework (no a blogs ni a vídeos) y la exposición no supera los 5 minutos.

Capítulo 2

Instalación del entorno de trabajo

Para empezar

1. ¿Qué hace falta tener instalado en un equipo para ejecutar un programa escrito en Python?
2. Si dos proyectos necesitan versiones distintas de la misma librería, ¿cómo pueden convivir en el mismo equipo?
3. ¿Por qué un framework se instala con un gestor de paquetes en lugar de descargarlo de una web?

2.1 Qué se va a instalar

Para trabajar con Django hacen falta tres piezas:

Pieza	Qué es
Python	El intérprete del lenguaje en el que está escrito Django y en el que se programará la aplicación
pip	El gestor de paquetes de Python: descarga e instala librerías y frameworks desde el repositorio oficial PyPI (*Python Package Index*)
Entorno virtual (`venv`)	Una carpeta con una copia aislada de Python y de los paquetes de **un solo proyecto**
Django	El framework, que se instala con pip dentro del entorno virtual

Se usará **Django 5.2**, que es una versión **LTS** (*Long Term Support*): recibe correcciones de seguridad hasta abril de 2028, lo que la hace adecuada para todo el curso. Django 5.2 funciona con Python 3.10 a 3.13; se instalará **Python 3.13**.

Todo se instala para el usuario de Windows, sin necesidad de permisos de administrador.

2.2 Instalación de Python

2.2.1 La terminal PowerShell

Los comandos de este documento se escriben en **PowerShell**. Para abrirla se pulsa `Win + X` y se elige **Terminal** (Windows 11) o **Windows PowerShell** (Windows 10). El indicador de la terminal tiene este aspecto:

```
PS C:\Users\usuario>
```

2.2.2 Comprobar si ya hay Python

```
> python --version
```

Resultado	Significado	Acción
`Python 3.10.x` a `Python 3.13.x`	Hay un Python válido	Continuar en el apartado 2.3
Se abre la Microsoft Store, o el comando no se reconoce	No hay Python, solo el acceso directo de la Store	Instalar Python
`Python 3.9` o anterior, o `Python 3.14` o posterior	Versión no recomendada para Django 5.2	Instalar Python 3.13 (pueden convivir varias versiones)

No se debe instalar Python desde la Microsoft Store: esa versión da problemas con las rutas y los permisos.

2.2.3 Descarga e instalación

1. Abrir https://www.python.org/downloads/windows/, buscar la última versión **Python 3.13.x** y descargar el **Windows installer (64-bit)**.
2. Ejecutar el instalador. En la primera pantalla:
   - Marcar **Add python.exe to PATH**. Es la opción más importante: permite usar el comando `python` desde cualquier carpeta.
   - Si no se tienen permisos de administrador, desmarcar **Use admin privileges when installing py.exe**.
   - Pulsar **Install Now**.
3. Cerrar la terminal y abrir otra nueva. La terminal que ya estaba abierta no detecta el Python recién instalado.

Ejercicio 2.1. Comprobación de la instalación de Python

Objetivo: confirmar que Python y pip están instalados y que Windows usa la versión correcta.

```
> python --version
> python -m pip --version
> where.exe python
```

Verificación. El primer comando muestra `Python 3.13.x`. El segundo muestra `pip 25.x from ...\Python313\...`. En la salida de `where.exe python`, la **primera línea** es `C:\Users\usuario\AppData\Local\Programs\Python\Python313\python.exe`.

Si la primera línea contiene `WindowsApps`, Windows sigue usando el acceso directo de la Microsoft Store. Se desactiva en *Configuración, Aplicaciones, Configuración avanzada de aplicaciones, Alias de ejecución de aplicaciones*, apagando los alias `python.exe` y `python3.exe`. Después se abre una terminal nueva.

2.3 Visual Studio Code

El editor del módulo es **Visual Studio Code**:

1. Descargar el instalador **User Installer** desde https://code.visualstudio.com/.
2. Durante la instalación, marcar **Agregar a PATH** y **Agregar la acción "Abrir con Code" al menú contextual**.
3. Instalar la extensión **Python** (de Microsoft) desde el panel de extensiones (`Ctrl + Mayús + X`).

2.4 La carpeta de trabajo y el entorno virtual

2.4.1 Por qué un entorno virtual

Si se instalan los paquetes directamente en el Python del sistema, todos los proyectos comparten las mismas versiones. Cuando un proyecto antiguo necesita Django 4 y uno nuevo Django 5, no pueden convivir. Un **entorno virtual** resuelve el problema: cada proyecto tiene su propia carpeta `.venv` con su propio Python y sus propios paquetes.

El entorno virtual **no se copia ni se entrega**: se reconstruye en cualquier equipo a partir de la lista de paquetes del proyecto, el fichero `requirements.txt`.

2.4.2 Carpeta de trabajo

Los proyectos se guardan en una carpeta **sin espacios ni tildes** y **fuera de OneDrive**. En muchos equipos las carpetas *Documentos* y *Escritorio* se sincronizan con OneDrive, y la sincronización interfiere con los entornos virtuales.

```
> mkdir $HOME\proyectos\pauf
> cd $HOME\proyectos\pauf
```

2.4.3 Permitir la activación del entorno en PowerShell

Activar un entorno virtual consiste en ejecutar un pequeño script, y PowerShell no permite ejecutar scripts por defecto. Se comprueba la política de ejecución del usuario:

```
> Get-ExecutionPolicy -Scope CurrentUser
```

Si el resultado es `Undefined` o `Restricted`, se cambia solo para el usuario actual:

```
> Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

`RemoteSigned` permite ejecutar los scripts creados en el propio equipo y exige firma digital a los descargados de Internet. Es el ajuste habitual en un equipo de desarrollo. Si el equipo no permite cambiarlo, el entorno se puede activar desde la terminal `cmd` con `.venv\Scripts\activate.bat`.

Ejercicio 2.2. Creación del entorno virtual e instalación de Django

Objetivo: crear un proyecto con su propio entorno virtual e instalar Django 5.2 en él.

1. Crear la carpeta del proyecto y entrar en ella:

   ```
   > mkdir hola_django
   > cd hola_django
   ```

2. Crear y activar el entorno virtual:

   ```
   > python -m venv .venv
   > .\.venv\Scripts\Activate.ps1
   ```

3. Con el entorno activado, actualizar pip e instalar Django. La expresión `~=5.2.0` significa «la última versión 5.2.x»:

   ```
   > python -m pip install --upgrade pip
   > python -m pip install "django~=5.2.0"
   ```

4. Guardar la lista exacta de paquetes instalados:

   ```
   > pip freeze > requirements.txt
   ```

Verificación.

- El indicador de la terminal empieza por `(.venv)`.
- `where.exe python` muestra en primer lugar una ruta que termina en `hola_django\.venv\Scripts\pytho`
- `python -m django --version` muestra `5.2.x`.

- `requirements.txt` contiene las líneas `asgiref==...`, `Django==5.2.x` y `sqlparse==...` (y en Windows, `tzdata==...`).

Cada vez que se abre una terminal nueva hay que volver a activar el entorno. Si el indicador no empieza por `(.venv)`, el entorno no está activado y los comandos usarán el Python del sistema, donde Django no está instalado.

2.5 Creación del proyecto Django

Django incluye la utilidad `django-admin`, que genera la estructura base de un proyecto:

```
> django-admin startproject config .
```

El primer argumento (`config`) es el nombre del paquete de configuración del proyecto. El **punto final** indica que el proyecto se cree en la carpeta actual; sin él, Django crea una carpeta adicional y el proyecto queda anidado (`config\config\`).

La estructura generada es la siguiente:

```
hola_django/
    .venv/              entorno virtual (no se entrega)
    config/
        __init__.py     convierte la carpeta en un paquete de Python
        settings.py     configuración del proyecto
        urls.py         tabla de rutas principal
        asgi.py         punto de entrada para servidores ASGI
        wsgi.py         punto de entrada para servidores WSGI
    manage.py           utilidad de administración del proyecto
    requirements.txt    lista de dependencias
```

Fichero	Función
`manage.py`	Ejecuta las órdenes de administración: arrancar el servidor (`runserver`), crear aplicaciones (`startapp`), aplicar migraciones (`migrate`)…
`config/settings.py`	Reúne la configuración: aplicaciones instaladas (`INSTALLED_APPS`), *middleware* (`MIDDLEWARE`), base de datos (`DATABASES`), idioma, zona horaria, modo de depuración (`DEBUG`)…
`config/urls.py`	Asocia cada ruta (URL) con el código que la atiende

Fichero	Función
`config/wsgi.py` y `config/asgi.py`	Permiten que un servidor web de producción ejecute la aplicación (se usarán en la Unidad 4)

Ejercicio 2.3. Arranque del servidor de desarrollo

Objetivo: crear el proyecto, preparar su base de datos y comprobar que Django funciona.

1. Crear el proyecto con `django-admin startproject config .` (con el punto final).
2. Crear las tablas que necesitan las aplicaciones incluidas de serie (usuarios, sesiones, administración). Django las guarda en una base de datos SQLite, el fichero `db.sqlite3`:

   ```
   > python manage.py migrate
   ```

3. Arrancar el servidor de desarrollo:

   ```
   > python manage.py runserver
   ```

4. Abrir http://127.0.0.1:8000 en el navegador.

Verificación. La terminal muestra `Starting development server at http://127.0.0.1:8000/` y el navegador muestra la página de bienvenida de Django con el mensaje «**The install worked successfully! Congratulations!**». El servidor se detiene con `Ctrl + C`.

2.6 Problemas frecuentes

Síntoma	Causa	Solución
El comando `python` abre la Microsoft Store	Alias de ejecución de la Store activos	Desactivarlos (Ejercicio 2.1)
`python` no se reconoce	No se marcó *Add python.exe to PATH*, o no se abrió una terminal nueva	Abrir otra terminal; si persiste, reinstalar marcando la opción
No se puede cargar `Activate.ps1` porque la ejecución de scripts está deshabilitada	Política de ejecución de PowerShell	`Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned`, o activar desde `cmd`
`django-admin` no se reconoce	Entorno virtual no activado	Activarlo, o usar `python -m django startproject config .`

Síntoma	Causa	Solución
`No matching distribution found for django~=5.2.0`	Python anterior a 3.10, o sin conexión	Comprobar `python --version` dentro del entorno
Aparece `config\config\`	Faltó el punto final en `startproject`	Borrar la carpeta y repetir el comando con el punto
`Error: That port is already in use.`	Ya hay otro servidor en el puerto 8000	Detenerlo, o usar `python manage.py runserver 8001`
El navegador no puede acceder al sitio	El servidor no está en marcha, o se escribió `https://`	Comprobar la terminal y usar `http://`

Para reflexionar. *El entorno virtual ocupa decenas de megabytes y no se entrega nunca. ¿Qué fichero permite a otra persona reconstruirlo exactamente igual, y por qué es mejor así?*

Capítulo 3

Primer proyecto: sin framework y con framework

Para empezar

1. ¿Qué tiene que hacer exactamente un programa para responder a una petición HTTP?
2. ¿Qué ocurre si se pide a un servidor una dirección que no existe?
3. ¿Cuántas líneas de código creéis que hacen falta para responder «Hola» en una dirección web?

3.1 La misma página, dos veces

Para comprobar en la práctica qué aporta un framework, se va a programar la misma funcionalidad de dos maneras: una dirección `/hola` que responde con el texto `Hola`, y un error 404 (*no encontrado*) para cualquier otra dirección.

3.2 Sin framework: el servidor de la biblioteca estándar

Python incluye en su biblioteca estándar el módulo `http.server`, que permite programar un servidor HTTP elemental. El programador recibe la petición y tiene que hacer todo lo demás: comparar la ruta, elegir el código de estado, escribir las cabeceras y codificar el cuerpo de la respuesta.

```
# sin_framework/servidor.py
from http.server import BaseHTTPRequestHandler, HTTPServer


class Manejador(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == "/hola":
            codigo, cuerpo = 200, "Hola"
        else:
            codigo, cuerpo = 404, "No encontrado"
        datos = cuerpo.encode("utf-8")
```

```
        self.send_response(codigo)
        self.send_header("Content-Type", "text/plain; charset=utf-8")
        self.send_header("Content-Length", str(len(datos)))
        self.end_headers()
        self.wfile.write(datos)


if __name__ == "__main__":
    servidor = HTTPServer(("127.0.0.1", 8000), Manejador)
    print("Servidor en http://127.0.0.1:8000  (Ctrl+C para parar)")
    servidor.serve_forever()
```

Elemento	Qué hace
`do_GET`	Método que el servidor llama cada vez que recibe una petición `GET`
`self.path`	Ruta pedida por el cliente (por ejemplo, `/hola`)
`send_response(codigo)`	Escribe la línea de estado: `200 OK`, `404 Not Found`…
`send_header(...)`	Añade una cabecera HTTP a la respuesta
`end_headers()`	Marca el final de las cabeceras
`wfile.write(datos)`	Envía el cuerpo de la respuesta, que debe estar en bytes

Ejercicio 3.1. Servidor HTTP sin framework

Objetivo: ejecutar un servidor programado a mano y analizar sus respuestas.

1. Crear la carpeta `sin_framework` dentro de `$HOME\proyectos\pauf` y, en ella, el fichero `servidor.py` con el código anterior.
2. Ejecutarlo con `python servidor.py`.
3. Desde otra terminal, hacer las peticiones con `curl.exe`. En PowerShell hay que escribir `curl.exe` y no `curl`, porque `curl` es un alias de otro comando de PowerShell:

   ```
   > curl.exe -i http://127.0.0.1:8000/hola
   > curl.exe -i http://127.0.0.1:8000/hola/
   > curl.exe -i http://127.0.0.1:8000/adios
   ```

4. Detener el servidor con `Ctrl + C` antes de continuar, porque Django usará el mismo puerto.

Verificación. La primera petición devuelve `HTTP/1.0 200 OK`, la cabecera `Content-Type: text/plain; charset=utf-8` y el cuerpo `Hola`. La segunda (`/hola/`, con barra final) y la tercera devuelven `HTTP/1.0 404 Not Found`: la comparación de rutas es literal.

3.3 Con framework: proyecto y aplicación Django

3.3.1 Proyecto y aplicaciones

En Django se distinguen dos niveles:

- El **proyecto** es la configuración global del sitio: ajustes, tabla de rutas principal y puntos de entrada. Se creó en el apartado anterior con `startproject`.
- Una **aplicación** es un módulo que agrupa una funcionalidad concreta (por ejemplo, la gestión de incidencias o la de usuarios), con sus modelos, vistas, rutas y plantillas. Un proyecto contiene una o varias aplicaciones, y una aplicación bien diseñada se puede reutilizar en otros proyectos.

Una aplicación se crea con `manage.py`:

```
> python manage.py startapp saludos
```

```
saludos/
    __init__.py
    admin.py          registro de modelos en el panel de administración
    apps.py           configuración de la aplicación
    migrations/       historial de cambios de la base de datos
        __init__.py
    models.py         modelos (datos)
    tests.py          pruebas automatizadas
    views.py          vistas (código que atiende las peticiones)
```

3.3.2 Rutas y vistas

Una **vista** es una función de Python que recibe un objeto `HttpRequest` con los datos de la petición y devuelve un objeto `HttpResponse`. Una **ruta** asocia una dirección con una vista. Cada aplicación define sus rutas en su propio fichero `urls.py`, y el proyecto las **incluye** en su tabla de rutas principal con la función `include()`. Así cada aplicación es independiente.

```
# saludos/views.py
from django.http import HttpResponse


def hola(request):
    return HttpResponse("Hola")
```

```
# saludos/urls.py
from django.urls import path

from . import views

urlpatterns = [
```

```
    path("hola/", views.hola, name="hola"),
]
```

```
# config/urls.py
from django.contrib import admin
from django.urls import include, path

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("saludos.urls")),
]
```

Las rutas de `path()` **no empiezan por barra**: se escribe `"hola/"`, no `"/hola/"`. El parámetro `name` da un nombre a la ruta para poder referirse a ella desde otras partes del código sin escribir la dirección.

Ejercicio 3.2. Primera aplicación Django

Objetivo: crear una aplicación con una vista y una ruta propias.

1. En el proyecto `hola_django`, con el entorno activado, crear la aplicación `saludos`.
2. Escribir la vista `hola` en `saludos/views.py`.
3. Crear el fichero `saludos/urls.py` con su lista `urlpatterns`.
4. Incluir las rutas de la aplicación en `config/urls.py`.
5. Arrancar el servidor con `python manage.py runserver` y hacer estas peticiones:

   ```
   > curl.exe -i http://127.0.0.1:8000/hola/
   > curl.exe -i http://127.0.0.1:8000/hola
   > curl.exe -i http://127.0.0.1:8000/adios
   ```

6. Abrir también http://127.0.0.1:8000/adios en el navegador.

Verificación.

- `/hola/` devuelve `HTTP/1.1 200 OK` con el cuerpo `Hola`.
- `/hola` (sin barra) devuelve `HTTP/1.1 301 Moved Permanently` con la cabecera `Location: /hola/`.
- `/adios` devuelve `HTTP/1.1 404 Not Found`. En el navegador, Django muestra una página de error que enumera las rutas que ha probado (`admin/` y `hola/`).

3.4 Comparación

Aspecto	Sin framework	Con Django
Líneas escritas por el desarrollador	Unas 20	Unas 8
Quién decide qué código atiende cada ruta	El desarrollador, con `if`	El framework, recorriendo `urlpatterns`
Cabeceras y codificación de la respuesta	A mano	Automáticas (`HttpResponse`)
Ruta con o sin barra final	Comparación literal	Redirección automática a la forma con barra
Página de error 404	La que se programe	Generada por el framework
Sesiones, usuarios, administración, base de datos	No existen	Incluidas en el proyecto desde el primer momento

Dos comportamientos de la tabla merecen una explicación:

- **La redirección de `/hola` a `/hola/`** la realiza un componente llamado `CommonMiddleware` cuando el ajuste `APPEND_SLASH` vale `True` (su valor por defecto): si una dirección no coincide con ninguna ruta pero sí coincidiría añadiéndole la barra final, Django redirige automáticamente. Los componentes *middleware* se estudiarán en la Unidad 2.
- **La página 404 con la lista de rutas** solo aparece porque el proyecto está en **modo de depuración** (`DEBUG = True` en `settings.py`). En un servidor de producción esa información revelaría la estructura interna de la aplicación, por lo que siempre se trabaja con `DEBUG = False` (Unidad 4).

Por último, conviene observar `config/settings.py`. Las listas `INSTALLED_APPS` y `MIDDLEWARE` muestran que el proyecto ya incluye, sin haber escrito nada, gestión de usuarios y autenticación (`django.contrib.auth`), sesiones (`django.contrib.sessions`), protección de formularios frente a ataques CSRF (`CsrfViewMiddleware`) y un panel de administración (`django.contrib.admin`). Son las tareas repetitivas del apartado 1.1.

> ***Para reflexionar.*** *En la versión sin framework, ¿quién llama al método `do_GET`? ¿Es ya una pequeña inversión de control? ¿Qué diferencia hay con lo que hace Django?*

Capítulo 4

Cierre de la unidad

4.1 Resumen

- Un **framework** proporciona la estructura base de una aplicación y resuelve las tareas que se repiten en casi todos los proyectos: rutas, sesiones, validación, base de datos, autenticación y gestión de errores.
- Su rasgo característico es la **inversión de control**: el framework dirige el flujo de ejecución y llama al código del desarrollador. Con una librería ocurre lo contrario.
- Los frameworks de servidor organizan la aplicación con el patrón **MVC** (Modelo, Vista, Controlador); Django lo llama **MTV** (Modelo, Plantilla, Vista).
- Cada capa de una aplicación web tiene sus frameworks. En el módulo se usarán Django, Vue, Docker, PrimeVue, pytest, Vitest y Playwright.
- Cada proyecto Python se trabaja en su propio **entorno virtual**, que se reconstruye a partir de `requirements.txt`.
- Un **proyecto Django** contiene la configuración global (`config/`) y una o varias **aplicaciones**; cada aplicación define sus vistas y sus rutas, que el proyecto incluye con `include()`.

4.2 Glosario

Término	Definición
Framework	Conjunto de componentes, convenciones y herramientas que proporciona la estructura base de una aplicación
Librería	Código reutilizable al que el programa llama cuando lo necesita
Inversión de control	Principio por el que el framework controla el flujo de ejecución y llama al código del desarrollador
MVC / MTV	Patrón que separa los datos (modelo), la presentación (vista o plantilla) y la coordinación (controlador o vista)

Término	Definición
LTS	*Long Term Support*: versión con soporte y correcciones de seguridad durante un periodo largo
pip	Gestor de paquetes de Python
Entorno virtual	Copia aislada de Python y de los paquetes de un proyecto
`requirements.txt`	Fichero con la lista exacta de dependencias de un proyecto Python
Proyecto Django	Configuración global de un sitio: ajustes, rutas principales y puntos de entrada
Aplicación Django	Módulo que agrupa una funcionalidad: modelos, vistas, rutas y plantillas
Vista (Django)	Función que recibe una petición (`HttpRequest`) y devuelve una respuesta (`HttpResponse`)
Ruta	Asociación entre una dirección (URL) y la vista que la atiende
`DEBUG`	Ajuste que activa el modo de depuración; nunca debe estar activo en producción

4.3 Preguntas de autoevaluación

1. Explicad con un ejemplo propio la diferencia entre una librería y un framework.
2. ¿Qué es la inversión de control? ¿Dónde se observa en el código de Django de esta unidad?
3. En el patrón MTV de Django, ¿qué componente equivale al controlador de MVC?
4. ¿Para qué sirve un entorno virtual? ¿Por qué no se entrega la carpeta `.venv`?
5. ¿Qué diferencia hay entre un proyecto y una aplicación en Django?
6. ¿Qué ficheros hay que modificar para añadir una nueva página a una aplicación Django?
7. ¿Por qué Django redirige `/hola` a `/hola/`? ¿Qué ajuste lo controla?
8. ¿Qué riesgo supone dejar `DEBUG = True` en un servidor de producción?

4.4 Referencias

- Documentación oficial de Django 5.2: https://docs.djangoproject.com/es/5.2/
- Tutorial oficial de Django, parte 1: https://docs.djangoproject.com/es/5.2/intro/tutorial01/
- Instalación de Django: https://docs.djangoproject.com/es/5.2/intro/install/
- Python en Windows: https://docs.python.org/es/3.13/using/windows.html
- Entornos virtuales (`venv`): https://docs.python.org/es/3.13/library/venv.html
- Módulo `http.server`: https://docs.python.org/es/3.13/library/http.server.html